

ADWAIT — AAROH AI/ML IMPLEMENTATION BRIEF

Project: AAROH

Problem: AI-Powered Dynamic Mental Health Monitoring and Distress Prediction System for Victims of Atrocities

Role: AI/ML, Model Training & Inference Owner

0. YOUR MISSION

You own the **complete AI/ML subsystem of AAROH**.

Your responsibility is to take interaction data and turn it into:

Interaction Data



Feature Extraction



Dynamic Distress Score



Longitudinal Trend



Escalation Prediction



Confidence / Uncertainty



Explanation



Versioned ML Prediction

The goal is **not** to create a generic sentiment-analysis model.

The goal is to build an ML system that can answer:

"Is this person's psychological distress worsening, and is there a meaningful probability of escalation within the defined prediction horizon?"

The existing AAROH backend already has a working **rule-based baseline**. Your job is to progressively replace/augment that baseline with genuine ML while preserving the working pipeline.

1. FIRST: UNDERSTAND THE EXISTING CODE

Before writing code, clone/pull the repository and inspect:

```
backend/  
├── models.py  
├── database.py  
├── features/  
├── risk/  
├── intervention_engine.py  
├── process_all_interventions.py  
└── validation/
```

Pay particular attention to:

```
features/text_features.py  
features/behavioural_features.py  
features/engagement_features.py  
features/longitudinal_features.py  
features/extract_features.py  
risk/risk_scoring.py  
risk/prediction.py  
risk/process_all_cases.py  
models.py
```

Run the existing pipeline **before changing anything**.

The current validation command is:

```
python validation/validate_pipeline.py
```

It must pass before you begin.

Also run the existing feature/risk tests where applicable.

Important

Do not assume that existing code is useless because it is rule-based.

It is the **baseline and integration contract**.

Your ML implementation should improve it rather than randomly replace working components.

2. CREATE YOUR BRANCH

Do not work directly on `main`.

Create:

```
git checkout main  
git pull origin main  
git checkout -b feature/adwait-ml
```

Your work stays on this branch until it is reviewed and merged.

3. YOUR COMPLETE OWNERSHIP

You own these areas:

ML/Data

- preprocessing
- text processing
- multilingual NLP
- behavioural features
- engagement features
- longitudinal features
- voice-derived features
- feature engineering
- feature validation

Modelling

- distress modelling
- escalation prediction

- temporal modelling
- calibration
- confidence estimation
- uncertainty
- abstention/human-review signals

Explainability

- prediction explanations
- contributing factors
- trend explanation
- baseline deviation explanation
- model/version metadata

Training

- dataset preparation
- training scripts
- validation split strategy
- experiments
- evaluation
- model selection
- checkpoints
- artifact generation

Inference

- loading trained models
- prediction pipeline
- preprocessing consistency
- model artifact management
- inference interface

Versioning

Every production-capable model must have:

model name

model version

training date

feature version

dataset/version identifier

evaluation metrics

4. WHAT YOU ARE NOT BUILDING

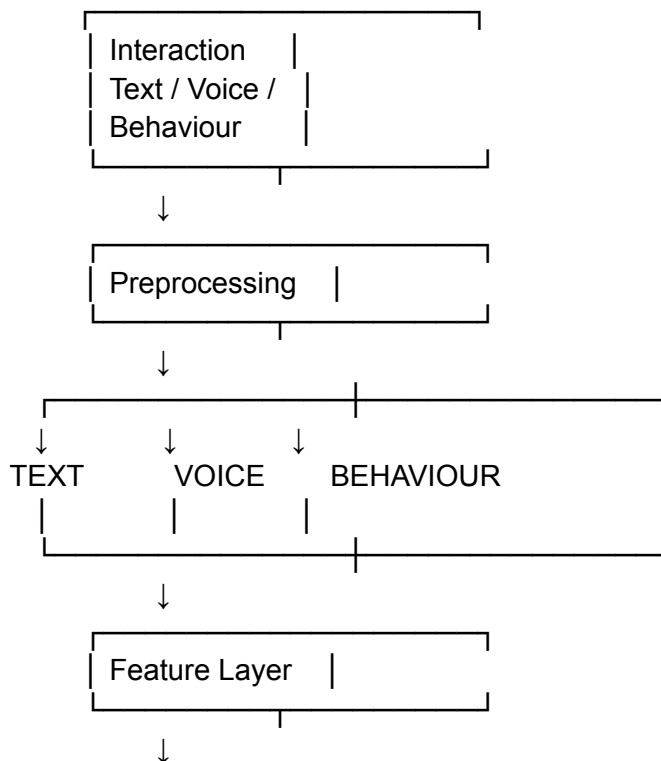
Do **not** build:

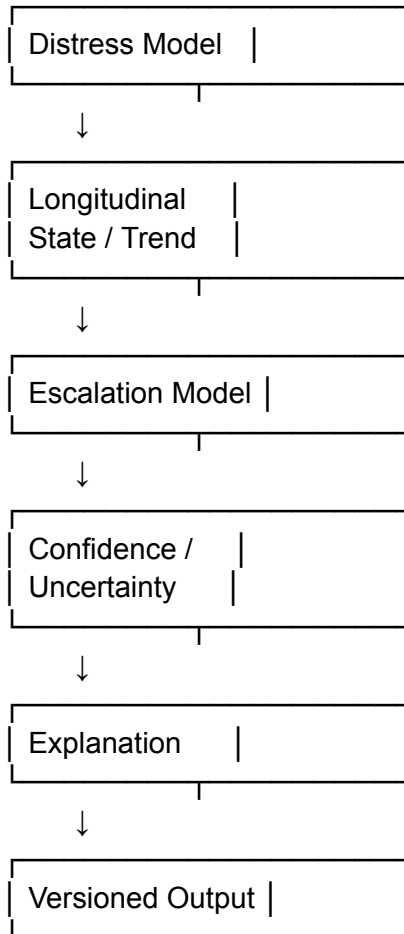
- FastAPI route architecture
- authentication
- frontend
- dashboards
- intervention assignment
- counsellor workflows
- cloud infrastructure
- production database administration
- fake NHAA APIs

You provide a clean ML interface that the application layer can consume.

5. TARGET ML ARCHITECTURE

Build toward:





6. STEP 1 — CLEAN ML MODULE STRUCTURE

Create a clean ML structure without destroying the existing feature/risk modules.

Target:

```
backend/
├── ml/
│   ├── __init__.py
│   ├── preprocessing/
│   ├── models/
│   ├── training/
│   ├── inference/
│   └── evaluation/
```



Each module should have one clear responsibility.

Do not create a giant `ml.py` containing everything.

7. STEP 2 — BUILD A PROPER FEATURE PIPELINE

The model should consume a structured feature representation.

At minimum, support:

Text

Potential features include:

- distress intensity
- fear
- intimidation indicators
- hopelessness
- isolation
- help-seeking language
- emotional state
- linguistic confidence

Behaviour

Examples:

- sleep disruption
- fear level
- social support
- safety response
- help requested

Engagement

Examples:

- missed check-in
- response delay
- engagement change
- declining response completion

Longitudinal

Examples:

- previous distress
- moving average
- recent change
- rate of change
- baseline deviation
- consecutive worsening observations
- volatility

Voice

Where consent and usable audio exist:

- speech rate
- pause ratio
- response latency
- pitch variability
- energy variation
- audio quality
- baseline deviation

Do not invent voice measurements from text.

8. STEP 3 — MULTILINGUAL TEXT INTELLIGENCE

AAROH cannot realistically be English-only.

Design the text pipeline so that the language field is respected.

The architecture should support Indian-language interactions.

Do **not** blindly translate everything into English and claim that makes the system multilingual.

Evaluate multilingual models that can handle relevant Indian languages.

Possible model families can include pretrained multilingual transformer architectures.

The important requirement is:

Use a pretrained multilingual model and fine-tune/evaluate it appropriately rather than attempting to train a giant language model from scratch.

9. STEP 4 — DEFINE THE ML TARGET

You must explicitly define what the model predicts.

Distress model

Produces a normalized distress representation.

For example:

0.00 → minimal

1.00 → very high

The exact formulation should be justified experimentally.

Escalation model

Predict:

probability that the person's distress will meaningfully escalate within the defined prediction horizon.

Current AAROH uses:

7 days

Keep the horizon configurable rather than hard-coding it throughout the ML code.

10. STEP 5 — DO NOT TRAIN ON FAKE CLAIMS

The current seed database contains synthetic/demo data.

That data is useful for:

- testing
- pipeline validation
- integration
- demonstrations

It is **not sufficient evidence for claiming that a clinically meaningful model has been trained.**

If genuine labelled data is unavailable:

1. build the ML pipeline;
2. use synthetic/demo data only for technical validation;
3. clearly document the limitation;
4. do not report fabricated accuracy;
5. design the training pipeline so legitimate datasets can later be substituted.

This is extremely important for the SIH presentation.

11. STEP 6 — START WITH A STRONG BASELINE MODEL

Do not immediately jump to a complicated neural network.

Build a reproducible baseline.

Depending on the final feature representation, candidates could include:

Logistic Regression
Random Forest
Gradient Boosting
XGBoost/LightGBM

Then compare against a more sophisticated model if justified.

For text:

TF-IDF + classical classifier

can be a useful baseline before transformer fine-tuning.

The objective is to demonstrate:

baseline → improved model → measured improvement.

Not:

"We used a transformer because AI."

12. STEP 7 — GOOGLE COLAB TRAINING WORKFLOW

Yes, **Google Colab** is appropriate for the heavier training work.

Use the following separation:

LOCAL COMPUTER

- |
- |— coding
- |— debugging
- |— unit tests
- |— preprocessing tests
- |— lightweight inference
- |— integration testing

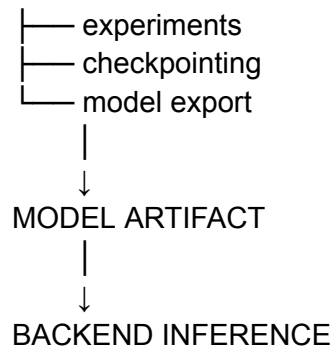


GITHUB



GOOGLE COLAB GPU

- |
- |— dataset preparation
- |— training
- |— validation



Important

Colab is a training environment, not the AAROH backend server.

The backend must never depend on:

"My Colab notebook is currently running."

After training:

model artifact
+
metadata
+
configuration

must be saved and loaded by the inference system.

Because Colab runtimes can terminate, **checkpoint regularly**.

Do not keep the only copy of a trained model inside an ephemeral runtime.

13. STEP 8 — TRAINING SCRIPT

Create reproducible training code.

For example:

```
ml/training/  
├── prepare_dataset.py  
├── train_distress.py  
└── train_escalation.py
```

```
|— evaluate.py
|— config.py
```

A training run should make it possible to determine:

Which data?
Which features?
Which model?
Which parameters?
Which version?
Which metrics?

Do not make training dependent on manually changing random notebook cells every time.

A Colab notebook may be used as a convenient execution layer, but the underlying training logic should live in reusable Python modules.

14. STEP 9 — DATA SPLITTING

Because AAROH is longitudinal, be extremely careful about leakage.

Do **not** randomly split individual interactions from the same person/case across train and test if that allows information from the same case to leak into both sets.

Prefer case/person-level separation where appropriate.

Document:

train
validation
test

and the reasoning behind the split.

If temporal generalization is being tested, also consider chronological evaluation.

15. STEP 10 — EVALUATE PROPERLY

Do not report only accuracy.

For escalation prediction, evaluate appropriate metrics such as:

Precision
Recall
F1
ROC-AUC
PR-AUC
Confusion Matrix
Calibration

For probabilistic predictions, calibration is particularly important.

Also examine:

- false negatives
- false positives
- performance across languages where data permits
- performance across relevant use cases where data permits
- performance under missing/noisy inputs

A model that says:

HIGH RISK

with 99% confidence should actually deserve that confidence.

16. STEP 11 — CONFIDENCE AND ABSTENTION

This is mandatory.

AAROH should not pretend to know something when the input is poor.

Examples:

Low-quality audio
+
Very short response
+
Missing historical data

may produce an unreliable prediction.

The ML system should be able to communicate:

prediction
confidence
data quality / uncertainty

Potential behaviour:

HIGH confidence
→ normal automated recommendation flow

LOW confidence
→ human review / insufficient evidence

The exact thresholds must be justified and configurable.

17. STEP 12 — EXPLAINABILITY

Every prediction should provide machine-readable explanation data.

Target structure:

```
{  
  "factors": [  
    "distress increased over recent interactions",  
    "fear indicators increased",  
    "engagement declined"  
  ],  
  "trend": "WORSENING",  
  "baseline_deviation": 0.31,  
  "model_version": "..."  
}
```

The explanation must correspond to actual model inputs/features.

Do **not** generate a beautiful paragraph with an LLM and call it explainability if it does not reflect the actual model.

18. STEP 13 — SHARED PREDICTION CONTRACT

Your ML subsystem should expose something conceptually equivalent to:

```
{
  "case_id": "CASE-001",
  "prediction_date": "2026-09-03",
  "escalation_probability": 0.78,
  "target_horizon_days": 7,
  "confidence": 0.86,
  "risk_level": "HIGH",
  "explanation": {
    "factors": [],
    "trend": "WORSENING",
    "baseline_deviation": 0.31,
    "model_version": "aaroH-escalation-v1"
  }
}
```

The final schema should be implemented cleanly and documented.

The application layer should be able to consume this without knowing how the model works internally.

19. STEP 14 — MODEL VERSIONING

Every exported model gets an explicit version.

Example:

```
aaroH-distress-v1
aaroH-escalation-v1
```

Metadata should include:

```
model_name
version
created_at
feature_version
```


training_dataset_version
training configuration
evaluation metrics

A model update should never silently overwrite the previous model.

20. STEP 15 — INFERENCE

Training and inference must be separate.

Target:

Training
↓
Saved Artifact
↓
Inference Loader
↓
Prediction

The inference process must work in a **fresh Python process**.

Test:

start application
↓
load model from artifact
↓
send valid input
↓
receive prediction

Do not rely on variables left over from a notebook.

21. STEP 16 — INTEGRATE WITH THE EXISTING RISK PIPELINE

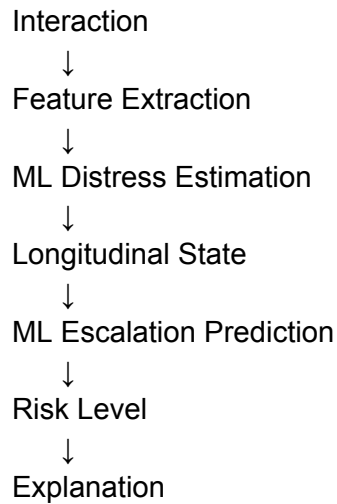
The existing pipeline already generates:

DistressState
Prediction

Your implementation should progressively integrate ML into that flow.

Do not delete the existing pipeline before the ML replacement has been tested.

The integration should ultimately become:



The existing rule-based implementation can remain as a baseline/fallback where appropriate, but the final architecture must make it explicit whether a result came from:

ML
baseline
fallback
insufficient evidence

Never silently present a rule-based number as an ML prediction.

22. STEP 17 — MISSING DATA

Real interactions will be incomplete.

Examples:

No voice
No text
Missed check-in

Partial response
Poor audio
Unknown language

The model must handle missingness intentionally.

Do not simply fill every missing value with zero without reasoning.

The system should preserve:

available data
missing data
data quality

as part of the prediction context.

23. STEP 18 — PRIVACY

Do not place real victim information into:

- GitHub
- public datasets
- Colab notebooks shared publicly
- screenshots
- debug logs
- training examples committed to the repository

For development, use:

synthetic data
de-identified/authorized data

as appropriate.

Never commit:

.env
API keys
database passwords
private datasets
credentials

24. STEP 19 — TESTING

You must create tests for:

Feature pipeline

valid input
missing input
bad input
multilingual input
short text
empty text

Model

prediction generated
probability in $[0,1]$
confidence in $[0,1]$
correct output schema

Longitudinal logic

Test:

stable
worsening
rapid worsening
improving
rapid improving

Inference

Test:

fresh process
model loading
prediction
invalid input
missing model artifact

Explainability

Ensure:

explanation exists

factors correspond to available features
model version exists

25. STEP 20 — VALIDATION GATE

Before requesting integration, all of these should work:

`python validation/validate_pipeline.py`

plus your ML tests.

Then test:

database

→ features

→ ML inference

→ prediction

→ existing pipeline

as one flow.

If your changes break an existing baseline test, fix the regression before handing it over.

26. YOUR GIT COMMITS

Use meaningful commits.

For example:

`feat(ml): add structured feature pipeline`

`feat(ml): add multilingual text baseline`

`feat(ml): add distress model training pipeline`

`feat(ml): add escalation model`

`feat(ml): add uncertainty estimation`

`feat(ml): add prediction explanations`

`feat(ml): add model versioning`

`test(ml): add inference validation`

Avoid:

final
final2
working
new
updated
changes

27. WHAT YOUR FINAL HANDOFF MUST CONTAIN

When your ML subsystem is ready, the repository should contain:

ML source code
training scripts
evaluation scripts
inference code
feature pipeline
model configuration
model metadata
tests
documentation
model artifacts or clearly documented artifact location

And documentation explaining:

- 1. What the model predicts**
- 2. What data it consumes**
- 3. What features it uses**
- 4. How it was trained**
- 5. How it was evaluated**
- 6. What the limitations are**
- 7. How confidence is calculated**
- 8. When it should abstain**

9. How to load the model

10. How another backend component calls inference

28. DEFINITION OF DONE

Your work is **not done** when:

"The model trains."

It is done when:

- [✓] Existing AAROH baseline understood
 - [✓] ML module cleanly structured
 - [✓] Feature pipeline reproducible
 - [✓] Multilingual text capability addressed
 - [✓] Behavioural features integrated
 - [✓] Engagement features integrated
 - [✓] Longitudinal features integrated
 - [✓] Voice features supported where available
 - [✓] Baseline model implemented
 - [✓] Improved model evaluated where justified
 - [✓] Proper train/validation/test strategy implemented
 - [✓] Data leakage addressed
 - [✓] Metrics documented
 - [✓] Confidence implemented
 - [✓] Uncertainty/abstention implemented
 - [✓] Explainability implemented
 - [✓] Model versioning implemented
 - [✓] Training reproducible
 - [✓] Colab GPU workflow documented
 - [✓] Model artifact export works
 - [✓] Fresh-process inference works
 - [✓] Prediction contract documented
 - [✓] Tests pass
 - [✓] Existing pipeline still passes
 - [✓] No secrets committed
 - [✓] No sensitive data committed
 - [✓] Limitations documented
 - [✓] ML subsystem ready for application integration
-

29. THE ORDER YOU SHOULD ACTUALLY FOLLOW

Do **not** attempt everything simultaneously.

Follow this order:

STEP 0

Understand repository + run baseline



STEP 1

Create ML structure



STEP 2

Clean/reproducible feature pipeline



STEP 3

Text + multilingual baseline



STEP 4

Behavioural + engagement features



STEP 5

Longitudinal representation



STEP 6

Voice feature integration



STEP 7

Distress model baseline



STEP 8

Escalation model baseline



STEP 9

Evaluate + fix leakage



STEP 10

Move heavier training to Colab GPU



STEP 11

Model improvement/fine-tuning



STEP 12
Confidence + uncertainty
↓
STEP 13
Explainability
↓
STEP 14
Model versioning
↓
STEP 15
Inference loader
↓
STEP 16
Integrate with AAROH risk pipeline
↓
STEP 17
End-to-end tests
↓
STEP 18
Documentation + handoff

ONE FINAL RULE

Do not optimize for the most impressive-looking model. Optimize for a model that AAROH can actually defend.

For the SIH demo, it is much stronger to say:

"We built a reproducible multilingual ML pipeline, evaluated it against a baseline, quantified uncertainty, and explicitly identify cases where the system should defer to human review."

than to say:

"We used a huge AI model and it gives 95% accuracy."

Build something that can survive questions from the judges.

AAROH — ADWAIT'S AI/ML CHECKLIST

Role: AI/ML, Model Training & Inference Owner

Branch: `feature/adwait-ml`

Rule: Don't move to the next phase until the current phase passes its checklist.

PHASE 0 — BEFORE TOUCHING CODE

Repository understanding

- ☐ Clone/pull the latest `main`
- ☐ Create `feature/adwait-ml`
- ☐ Read `README` / project documentation
- ☐ Read `models.py`
- ☐ Read `database.py`
- ☐ Read everything inside `features/`
- ☐ Read everything inside `risk/`
- ☐ Read `validation/validate_pipeline.py`
- ☐ Understand how `Case` connects to `Interaction`
- ☐ Understand how `Interaction` becomes `DistressState`
- ☐ Understand how `Prediction` is currently generated
- ☐ Understand how interventions currently consume prediction/risk information

Baseline

- ☐ Run the existing validation
- ☐ Confirm `AAROH PIPELINE VALIDATION: PASSED`
- ☐ Run existing feature tests
- ☐ Run existing risk tests
- ☐ Record the baseline result before modifying anything

Gate:

- ☐ I understand the existing pipeline well enough to explain it without opening the code.
-

PHASE 1 — ML ARCHITECTURE

- ☐ Create `backend/ml/`
- ☐ Create clear submodules:
 - ☐ `preprocessing/`
 - ☐ `models/`
 - ☐ `training/`
 - ☐ `inference/`
 - ☐ `evaluation/`
 - ☐ `explainability/`
 - ☐ `versioning/`
- ☐ Decide what belongs in `features/` vs `ml/`
- ☐ Document the ML pipeline
- ☐ Define input/output contracts
- ☐ Define model version format
- ☐ Define prediction horizon configuration
- ☐ Define confidence representation
- ☐ Define what happens when the model cannot make a reliable prediction

Gate:

- ☐ Another developer can understand the ML architecture without asking me.
-

PHASE 2 — FEATURE PIPELINE

Text

- ☐ Existing text features understood
- ☐ Text preprocessing implemented cleanly
- ☐ Empty text handled
- ☐ Very short text handled
- ☐ Noisy text handled
- ☐ Language identified/respected
- ☐ Multilingual architecture established

Behaviour

- ☐ Fear
- ☐ Sleep disruption
- ☐ Social support
- ☐ Safety response
- ☐ Help-seeking
- ☐ Other meaningful behavioural indicators

Engagement

- ☐ Response delay
- ☐ Missed check-ins
- ☐ Response completion
- ☐ Engagement change

Longitudinal

- ☐ Previous distress
- ☐ Recent distress average
- ☐ Change from previous observation
- ☐ Rate of change
- ☐ Baseline deviation
- ☐ Consecutive worsening
- ☐ Volatility

Voice

Where voice data and appropriate consent exist:

- ☐ Speech rate
- ☐ Pause ratio
- ☐ Response latency
- ☐ Pitch variability
- ☐ Energy variation
- ☐ Audio quality
- ☐ Baseline deviation
- ☐ Missing voice handled correctly
- ☐ Poor-quality audio handled correctly
- ☐ Voice features never fabricated

Gate:

- ☐ Same input produces the same feature representation reproducibly.
-

PHASE 3 — DATASET

- ☐ Define training dataset structure
- ☐ Define target labels
- ☐ Define prediction horizon
- ☐ Identify usable features
- ☐ Separate training/validation/test data
- ☐ Prevent case/person leakage
- ☐ Document dataset version
- ☐ Document missing-data handling
- ☐ Document preprocessing
- ☐ Document label-generation methodology

Synthetic data check

- ☐ Clearly mark synthetic/demo data
- ☐ Never present synthetic results as clinical evidence
- ☐ Never fabricate labels
- ☐ Never fabricate accuracy

Gate:

- ☐ I can explain exactly what one training row represents.
-

PHASE 4 — TEXT MODEL

- ☐ Build a simple baseline first
- ☐ Evaluate TF-IDF/classical baseline where appropriate
- ☐ Select multilingual pretrained model(s)
- ☐ Test relevant Indian-language capability
- ☐ Define fine-tuning strategy
- ☐ Define preprocessing/tokenization
- ☐ Train baseline

- ☐ Evaluate baseline
- ☐ Record metrics
- ☐ Compare improved model against baseline

Do NOT:

- ☐ Train a giant language model from scratch
- ☐ Claim multilingual capability without testing it
- ☐ Claim clinical validity without appropriate evidence

Gate:

- ☐ I have evidence showing why the selected model is preferable to the baseline.
-

PHASE 5 — DISTRESS MODEL

- ☐ Define distress target mathematically
- ☐ Define feature inputs
- ☐ Train baseline model
- ☐ Evaluate
- ☐ Test missing data
- ☐ Test noisy data
- ☐ Test edge cases
- ☐ Save model artifact
- ☐ Save metadata
- ☐ Record metrics
- ☐ Version model

Example:

aaroh-distress-v1

Gate:

- ☐ Given valid features, the model produces a reproducible distress estimate.
-

PHASE 6 — LONGITUDINAL MODEL

This is one of the **most important parts of AAROH**.

Don't treat every interaction as an isolated prediction.

- ☐ Calculate previous-state information
- ☐ Calculate baseline
- ☐ Calculate recent trend
- ☐ Calculate rate of change
- ☐ Detect sustained worsening
- ☐ Detect improvement
- ☐ Handle irregular interaction intervals
- ☐ Handle missing observations
- ☐ Test stable cases
- ☐ Test worsening cases
- ☐ Test rapidly worsening cases
- ☐ Test improving cases

Verify:

Stable → STABLE

Gradually worsening → WORSENING

Rapid worsening → RAPIDLY_WORSENING

Improving → IMPROVING

Gate:

- ☐ The model understands **change over time**, not just the latest interaction.

PHASE 7 — ESCALATION MODEL

Define exactly what:

"Escalation within 7 days"

means for the model.

Then:

- ☐ Define target
- ☐ Build baseline classifier
- ☐ Train model

- ☐ Evaluate
- ☐ Check class imbalance
- ☐ Check leakage
- ☐ Test false negatives
- ☐ Test false positives
- ☐ Evaluate calibration
- ☐ Compare models
- ☐ Save best justified model
- ☐ Version model

Example:

aaroh-escalation-v1

Gate:

- ☐ Probability output is meaningful and reproducible.
-

PHASE 8 — GOOGLE COLAB GPU

Local machine

Use local CPU for:

- ☐ Coding
- ☐ Debugging
- ☐ Unit tests
- ☐ Data pipeline testing
- ☐ Small experiments
- ☐ Lightweight inference

Colab

Use GPU for:

- ☐ Transformer fine-tuning
- ☐ Heavier model training
- ☐ Hyperparameter experiments
- ☐ Larger batch experiments

Before training

- ☐ Code committed to GitHub
- ☐ Dataset prepared
- ☐ Dataset version recorded
- ☐ Training configuration saved
- ☐ Random seed controlled where appropriate
- ☐ Checkpoint strategy established

During training

- ☐ Save checkpoints
- ☐ Save best model
- ☐ Record metrics
- ☐ Record configuration
- ☐ Record model version
- ☐ Do not rely on notebook state

After training

- ☐ Export model
- ☐ Export tokenizer/preprocessor if required
- ☐ Export metadata
- ☐ Store artifact safely
- ☐ Verify artifact can be loaded outside Colab

Gate:

- ☐ I can shut down Colab completely and still run AAROH inference locally.
-

PHASE 9 — MODEL EVALUATION

For classification:

- ☐ Precision
- ☐ Recall
- ☐ F1
- ☐ ROC-AUC where appropriate
- ☐ PR-AUC where appropriate
- ☐ Confusion matrix

For probabilities:

- ☐ Calibration checked
- ☐ Confidence behaviour checked

Also inspect:

- ☐ False negatives
- ☐ False positives
- ☐ Missing data
- ☐ Noisy data
- ☐ Language differences where data permits
- ☐ Relevant use cases where data permits

Gate:

- ☐ I can explain where the model performs well **and where it fails**.
-

PHASE 10 — CONFIDENCE + ABSTENTION

Implement:

- ☐ Prediction confidence
- ☐ Input/data quality consideration
- ☐ Uncertainty representation
- ☐ Low-confidence detection
- ☐ Abstention/human-review signal
- ☐ Configurable thresholds

Example:

Prediction: HIGH

Probability: 0.78

Confidence: LOW

should **not** be treated identically to:

Prediction: HIGH

Probability: 0.78

Confidence: HIGH

Gate:

- ☐ The system can explicitly say: **"I don't have enough reliable evidence."**
-

PHASE 11 — EXPLAINABILITY

Every prediction should be explainable through actual model inputs.

- ☐ Top contributing factors
- ☐ Current trend
- ☐ Baseline deviation
- ☐ Relevant recent changes
- ☐ Model version
- ☐ Explanation generated from actual features
- ☐ No fabricated explanation

Example:

```
{  
  "factors": [  
    "distress increased across recent interactions",  
    "fear indicators increased",  
    "engagement declined"  
  ],  
  "trend": "WORSENING",  
  "baseline_deviation": 0.31,  
  "model_version": "aaroH-escalation-v1"  
}
```

Gate:

☐ If someone asks "Why did the model produce this?", I can answer using actual model evidence.

PHASE 12 — MODEL VERSIONING

For every model:

- ☐ Name
- ☐ Version
- ☐ Creation date
- ☐ Feature version
- ☐ Dataset version
- ☐ Training configuration
- ☐ Evaluation metrics

Example:

Model:

aaroh-escalation-v1

Dataset:

demo-dataset-v1

Features:

feature-schema-v2

- ☐ Never silently overwrite an existing model
 - ☐ Never deploy an unidentified artifact
-

PHASE 13 — INFERENCE

- ☐ Create inference loader
- ☐ Load model artifact
- ☐ Load required tokenizer/preprocessor
- ☐ Validate input
- ☐ Generate prediction
- ☐ Generate confidence
- ☐ Generate explanation
- ☐ Attach model version
- ☐ Handle invalid input
- ☐ Handle missing artifact
- ☐ Handle insufficient data

Critical test

Kill Python.

Start a completely fresh process.

Then:

load model

↓

provide input

↓

generate prediction

- ☐ Works without notebook state
- ☐ Works without training code
- ☐ Works without Colab

Gate:

- ☐ Fresh-process inference works.



PHASE 14 — AAROH INTEGRATION

Connect:

Interaction



Features



ML



Distress State



Longitudinal State



Escalation Prediction

Check:

- ☐ Case ID preserved
- ☐ Interaction date preserved
- ☐ Prediction date correct
- ☐ Probability valid
- ☐ Confidence valid
- ☐ Risk level valid
- ☐ Explanation present
- ☐ Model version present
- ☐ Existing pipeline still works



PHASE 15 — SAFETY CHECK

Before handoff:

- ☐ No autonomous medical diagnosis
- ☐ No autonomous legal decision
- ☐ No autonomous witness-protection decision
- ☐ No automatic punitive action

- ☐ Low confidence can reach human review
 - ☐ Consent constraints respected
 - ☐ Voice analysis only when permitted
 - ☐ Sensitive information not unnecessarily exposed
 - ☐ No raw victim data in GitHub
 - ☐ No API keys in code
 - ☐ No passwords in notebooks
-

PHASE 16 — FINAL TEST

Run everything.

Existing system

- ☐ Database tests pass
- ☐ Feature tests pass
- ☐ Risk tests pass
- ☐ Pipeline validation passes

ML

- ☐ Feature tests pass
- ☐ Training pipeline works
- ☐ Evaluation works
- ☐ Model loads
- ☐ Inference works
- ☐ Confidence works
- ☐ Explainability works
- ☐ Versioning works

End-to-end

Case

↓

Interaction

↓

Features



ML



Distress



Trend



Escalation



Confidence



Explanation

- ☐ Complete flow works
- ☐ No broken existing functionality
- ☐ No hard-coded local paths
- ☐ No secrets
- ☐ No sensitive test data



FINAL HANDOFF CHECKLIST

Before creating the PR:

Code

- ☐ Clean ML directory
- ☐ No unnecessary files
- ☐ No notebooks containing the only implementation
- ☐ No debugging code

- ☐ No hard-coded paths

Models

- ☐ Model artifacts available/documented
- ☐ Versioned
- ☐ Metadata available
- ☐ Evaluation results documented

Documentation

- ☐ Architecture documented
- ☐ Training instructions documented
- ☐ Colab instructions documented
- ☐ Inference instructions documented
- ☐ Model limitations documented
- ☐ Dataset limitations documented

Git

- ☐ Changes committed
- ☐ Commit messages meaningful
- ☐ Branch up to date
- ☐ No secrets
- ☐ No sensitive datasets

PR

- ☐ Explain what changed
- ☐ Explain why
- ☐ List files changed
- ☐ List dependencies added
- ☐ List tests run
- ☐ Include evaluation results
- ☐ Mention limitations
- ☐ Mention integration requirements



RED-FLAG CHECKLIST

STOP and fix it if any of these happen:

- **✗** Model accuracy was invented
- **✗** Synthetic data presented as real-world evidence
- **✗** Train/test leakage exists
- **✗** Model output is called "clinical diagnosis"
- **✗** Rule-based score is presented as ML output
- **✗** Confidence is just a random number
- **✗** Explanation doesn't correspond to model features
- **✗** Colab notebook is required for production inference
- **✗** Model cannot load in a fresh process
- **✗** API keys are committed
- **✗** Real victim data is committed
- **✗** Existing AAROH functionality was deleted unnecessarily
- **✗** A giant model is being trained from scratch without a very strong reason